

Alexandria University

Faculty of Engineering
Mechatronics and Robotics Department

Metal Discrimination System Report

Course: Microcontroller Engineering 1 (EEP217)

Submitted to: Dr. Hossam Abdelbaki

Prepared By:

Yassa Emad Boshra	10395
Mina Naim Shaker	9754

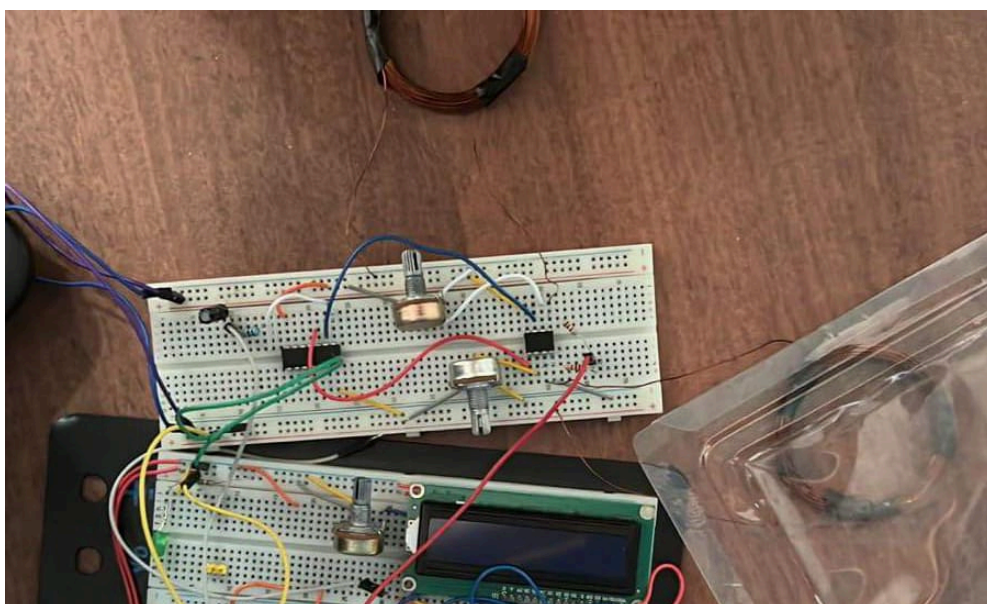
1. Introduction & Operating Principle

This project presents the design, implementation, and analysis of an inductive metal discriminator system centered around the ATmega32A microcontroller. The primary objective is to sense the presence of metallic objects and differentiate between ferrous and non-ferrous materials based on electromagnetic induction principles.

The core topology relies on an adapted Induction Balance (IB) differential configuration. Traditional single-coil induction systems suffer from significant thermal and environmental drift, which alters the intrinsic resistance and inductance of the sensing loops over time, leading to false positives and degraded resolution. To overcome this limitation, this architecture implements a dual-coil approach:

- **Sense Coil (Search Loop):** Exposed directly to the external environment and any target metallic objects.
- **Reference Coil:** Completely isolated and shielded from external metallic interference, acting as a baseline environmental compensator.

When an alternating excitation signal is applied across both loops, the presence of an external metal near the Sense Coil distorts its local electromagnetic field. This distortion manifests as variations in eddy current reflections, altering both the amplitude and the phase angle of the coil's response relative to the isolated Reference Coil. By comparing these two signals via differential hardware structures, the environmental baseline drifts are canceled out, leaving only the pure delta induced by the target metal.

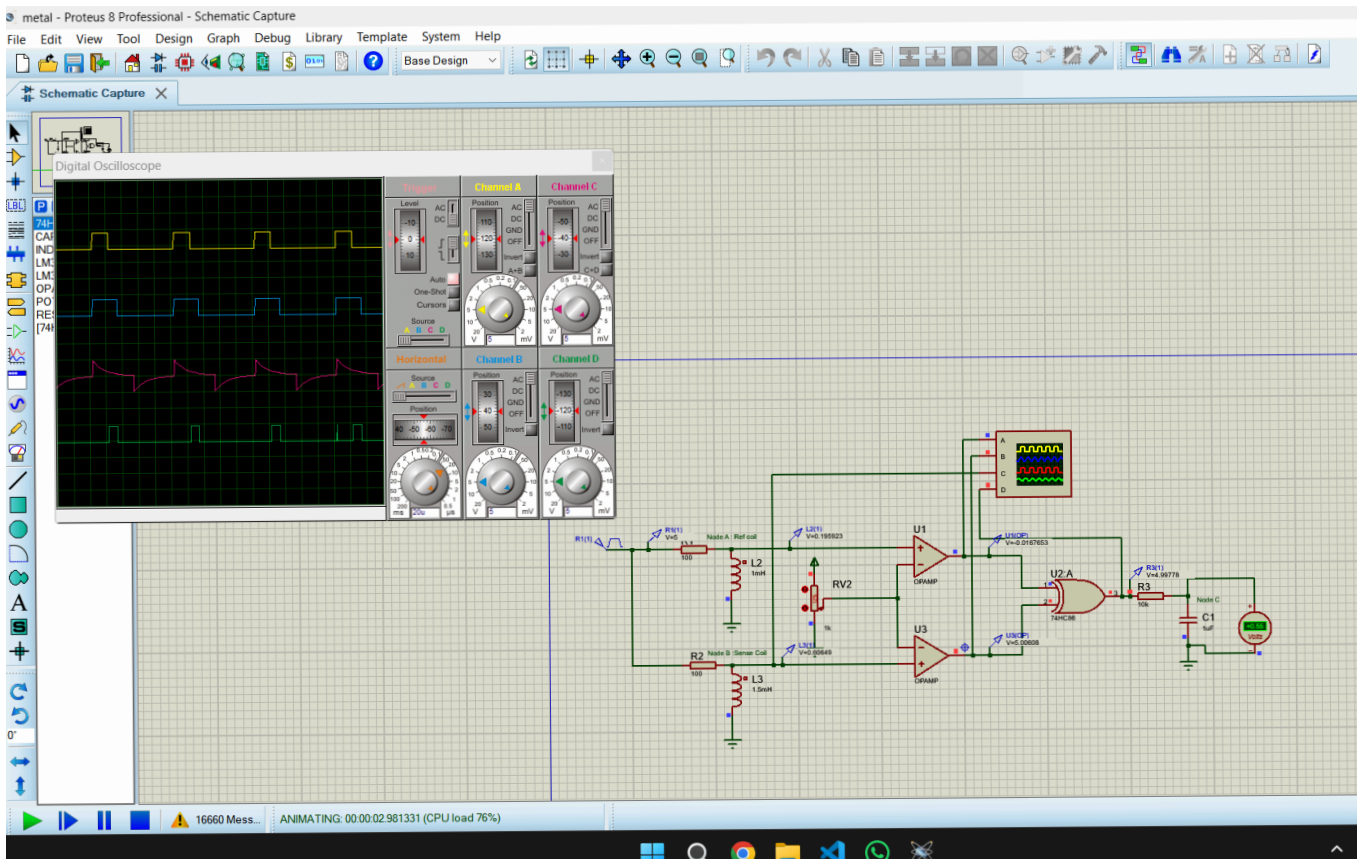
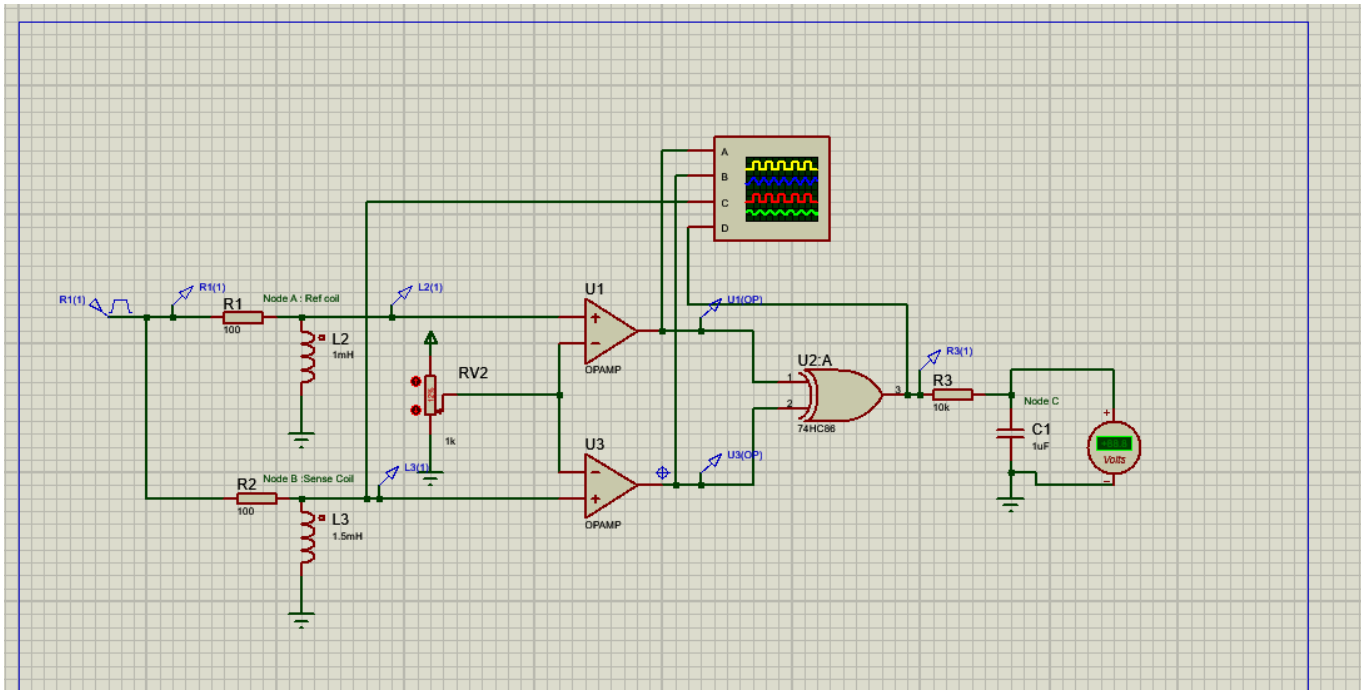


2. System Components & Hardware Specifications

The system utilizes standard, highly reliable analog and digital integrated components configured to acquire, amplify, and mix the high-frequency response waveforms. The component parameters and their functional roles are structured below:

Microcontroller	ATmega32A	Generates excitation pulses and samples output voltage signals via ADC.
Coils (x2)	N = 64 turns, Diameter = 5 cm, Wire = 0.5 mm, Inductance $\approx$ 1.0–1.5 mH	Sensing elements configured as Reference Coil (L2) and Sense Coil (L3).
Operational Amplifier	LM358 (Dual Channel Op-Amp)	Configured as high-gain open-loop comparators to square the sinusoidal waveforms into sharp edges.
XOR Logic Gate	74LS86	Acts as a hardware phase detector comparing Reference vs. Sense channels.
Potentiometers (x2)	1 k Ω	Used for precise analog balancing, tuning thresholds, and calibration offsets.
Low-Pass Filter Resistor	10 k Ω (R3)	Integrator resistor working with C1 to convert PWM/XOR pulses to DC levels.
Fixed Resistors	1 k Ω (x2)	Inline current limiting and biasing networks for balancing nodes.
Filter Capacitor	1 μ F (C1)	Integrator capacitor forming an RC low-pass

3. Simulation circuit and Output



4. Signal Processing & Limitations Analysis

During empirical testing, an essential architectural limitation was observed regarding high-resolution discrimination between specific non-ferrous metals, notably Copper and Aluminum. While the system identifies metal presence with high sensitivity, precise classification remains challenging.

The fundamental cause stems from the excitation source: the ATmega32A outputs a digital PWM (Square Wave) signal rather than a pure, single-frequency Sinusoidal Wave. A square wave contains a rich series of odd harmonics determined by its Fourier series components:

$$V(t) = (4/\pi) * [\sin(\omega t) + (1/3)*\sin(3\omega t) + (1/5)*\sin(5\omega t) + ...]$$

When these high-order harmonics (3ω , 5ω , etc.) pass through the inductive networks, each harmonic undergoes a distinct phase shift and attenuation factor, because inductive reactance is directly proportional to frequency ($X_L = 2\pi fL$). Consequently, the waveform emerging from the coils is highly distorted, causing overlapping phase responses at the XOR detector phase-demodulation stage. This overlapping blinds the phase detector from differentiating subtle variations between high-conductivity non-ferrous metals like copper and aluminum.

To overcome this constraint without adding complex external analog oscillators (like a standalone function generator), the firmware architecture was designed to accommodate two separate evaluation techniques:

1. Phase Shift Method: Compares the zero-crossing alignment of the reference and sense loops via the XOR phase detector. Ideally suited for distinguishing between highly magnetic (ferrous) and highly conductive (non-ferrous) categories.

2. Amplitude Method: Measures the raw voltage drop and attenuation over the sense network directly via the ATmega32A ADC. Because different metals present unique eddy current losses, checking the amplitude envelope provides a more reliable metric under square wave excitation.

Currently, the system is actively operated using the Amplitude Method firmware, which delivers superior stability and reproducible classification under the present square wave constraints.

5. Technical Roadblocks, Real-World Bugs & Optimization Matrix

5.1. Imprecise Discrimination Between Iron (Steel) and Aluminum

• **The Problem:** Both materials return a negative-going delta signal relative to the calibrated null baseline. For example, Steel registers at approximately -80 units, while Aluminum registers at approximately -40 units. The system is predominantly reading the absolute magnitude of electromagnetic coupling (attenuation intensity) rather than a clear directional phase sign vector.

• **Engineering Solutions:**

- Sweep and test at multiple distinct excitation frequencies to identify material-specific impedance curves.
- Invert the physical winding orientation or wiring terminals of the Sense Coil relative to the Reference Coil to shift the phase baseline into a highly responsive quadrant.
- Replace the hardware XOR gate with an active analog Phase Detector chip capable of preserving quadrant sign information.

5.2. Excitation Frequency Disalignment from the Ideal Splitting Point

• **The Problem:** The baseline operating frequency of 10 kHz does not fall on the ideal crossover threshold where phase characteristics invert cleanly. At the ideal frequency, magnetic characteristics dominate Ferrous materials (causing a phase lag), whereas eddy currents dominate highly conductive materials like Copper/Aluminum (causing a phase lead).

• **Engineering Solutions:** Systematically adjust the Output Compare Register 0 (OCR0) of the ATmega32 8-bit Timer0 to sweep the frequency range. By evaluating fixed decrements, the ideal cross-over point can be isolated:

- OCR0 = 90 (Lower Frequency band)
- OCR0 = 80
- OCR0 = 70
- OCR0 = 60 (Higher Frequency band)

The optimal register setting is reached when Iron consistently drives the signal in one polarity direction while Copper/Aluminum drives it in the opposite direction.

5.3. Functional Bottlenecks of the XOR Phase Detector Topology

- **The Problem:** The 74HC86 XOR gate operates strictly as a magnitude phase-difference indexer. It generates a duty cycle proportional to the absolute difference $|\Delta\theta|$ between the inputs, but it cannot differentiate whether the Sense channel is leading or lagging the Reference channel. This causes a loss of the phase sign vector, making stable material assignment erratic.
- **Advanced Engineering Solutions:** To elevate the architecture to a fully professional standard, the XOR configuration should be replaced with:
 - A Synchronous Demodulator / Detector circuit.
 - An active analog Mixer/Multiplier Detector topology.
 - An integrated AD630 High-Precision Balanced Modulator/Demodulator chip to capture true phase quadrant vectors.

6. Codes

Option A: Amplitude-Based Processing Firmware (Currently Deployed)

```
#define F_CPU 11059200UL

#include <avr/io.h>

#include <util/delay.h>

#include <stdio.h>

#include <stdlib.h>

#include "my_lcd.h"

// State Machine

typedef enum {

    STATE_CLEAR = 0,

    STATE_FERROUS,

    STATE_NON_FERROUS

} DetectionState;

// PWM Init - 10kHz

void PWM_10kHz_Init() {

    DDRB |= (1 << PB3);

    TCCR0 = (1 << WGM01) | (1 << COM00) | (1 << CS01);

    OCRO = 68;

}
```

```

// ADC Init

void ADC_Init() {

    ADMUX = (1 << REFS0);

    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);

}

uint16_t ADC_Get_Value() {

    uint32_t sum = 0;

    for (uint16_t i = 0; i < 256; i++) {

        ADCSRA |= (1 << ADSC);

        while (ADCSRA & (1 << ADSC));

        sum += ADCW;

    }

    return (uint16_t)(sum / 256);

}

// Baseline (Drift Compensation)

void update_baseline(uint16_t *base, uint16_t current) {

    *base = (*base * 99 + current) / 100;

}

int main(void) {

    uint16_t base_value = 0;

    uint16_t current_value = 0;

    uint16_t filtered_value = 0;

    char buffer[16];

    // Thresholds

    const uint16_t THRESHOLD_ENTER = 8;

    const uint16_t THRESHOLD_EXIT = 12;

    const uint8_t CONFIRM_COUNT = 4;

    uint8_t confirm_counter = 0;

    DetectionState current_state = STATE_CLEAR;

    DetectionState pending_state = STATE_CLEAR;

```



```

PWM_10kHz_Init();

ADC_Init();

DDRD &= ~(1 << PD2);

PORTD |= (1 << PD2);

LCD_Init();

LCD_Clear();

//Calibration

LCD_String("Calibrating...");

_delay_ms(2000);

uint32_t cal_sum = 0;

for (uint8_t i = 0; i < 32; i++) { // ????? ?? 16 ?? 32

    cal_sum += ADC_Get_Value();

    _delay_ms(50);

}

base_value = (uint16_t)(cal_sum / 32);

filtered_value = base_value;

LCD_Clear();

while (1) {

    if (!(PIND & (1 << PD2))) {

        base_value = filtered_value;

        LCD_Clear();

        LCD_Gotoxy(0, 0);

        LCD_String("Zeroing...");

        _delay_ms(500);

        LCD_Clear();

    }

    current_value = ADC_Get_Value();

    // IIR Filter

    filtered_value = (filtered_value * 5 + current_value * 5) / 100;

    int16_t diff = (int16_t)filtered_value - (int16_t)base_value;

```

```

DetectionState new_pending;

// Ferrous
if (diff < -60) {

    new_pending = STATE_FERROUS;

}

// Non-Ferrous
else if (diff < -15) {

    new_pending = STATE_NON_FERROUS;

}

// Clear
else if (abs(diff) < THRESHOLD_EXIT) {

    new_pending = STATE_CLEAR;

}

// Keep Previous
else {

    new_pending = current_state;

}

if (new_pending == pending_state) {

    if (confirm_counter < CONFIRM_COUNT) confirm_counter++;

} else {

    pending_state = new_pending;

    confirm_counter = 1;

}

if (confirm_counter >= CONFIRM_COUNT) {

    current_state = pending_state;

}

if (current_state == STATE_CLEAR) {

    update_baseline(&base_value, filtered_value);

}

sprintf(buffer, "D:%-4d B:%-4u ", diff, base_value);

```

```

    LCD_Gotoxy(0, 0);

    LCD_String(buffer);

    LCD_Gotoxy(1, 0);

    switch (current_state) {

        case STATE_FERROUS:

            LCD_String("Iron/Steel Det !");

            break;

        case STATE_NON_FERROUS:

            LCD_String("Copper/Alum Det!");

            break;

        default:

            LCD_String("Clear...  ");

            break;

    }

    _delay_ms(50);

}

return 0;

}

```

Option B: Phase-Shift Discrimination Firmware (Experimental Reference)

```

#define F_CPU 11059200UL

#include <avr/io.h>

#include <util/delay.h>

#include <stdio.h>

#include <stdlib.h>

#include "my_lcd.h"

// State Machine

typedef enum {

    STATE_CLEAR = 0,

    STATE_FERROUS,

    STATE_NON_FERROUS

```

```

} DetectionState;

// PWM Init - 10kHz

void PWM_10kHz_Init() {

    DDRB |= (1 << PB3);

    TCCR0 = (1 << WGM01) | (1 << COM00) | (1 << CS01);

    OCRO = 68;

}

// ADC Init

void ADC_Init() {

    ADMUX = (1 << REFS0);

    ADCSRA = (1 << ADEN) | (1 << ADPS2) | (1 << ADPS1) | (1 << ADPS0);

}

uint16_t ADC_Get_Value() {

    uint32_t sum = 0;

    for (uint16_t i = 0; i < 256; i++) {

        ADCSRA |= (1 << ADSC);

        while (ADCSRA & (1 << ADSC));

        sum += ADCW;

    }

    return (uint16_t)(sum / 256);

}

// Baseline (Drift Compensation)

void update_baseline(uint16_t *base, uint16_t current) {

    *base = (*base * 99 + current) / 100;

}

int main(void) {

    uint16_t base_value = 0;

```

```

uint16_t current_value = 0;

uint16_t filtered_value = 0;

char buffer[16];

// Thresholds

const uint16_t THRESHOLD_ENTER = 8;

const uint16_t THRESHOLD_EXIT = 12;

const uint8_t CONFIRM_COUNT = 4;

uint8_t confirm_counter = 0;

DetectionState current_state = STATE_CLEAR;

DetectionState pending_state = STATE_CLEAR;

PWM_10kHz_Init();

ADC_Init();

DDRD &= ~(1 << PD2);

PORTD |= (1 << PD2);

LCD_Init();

LCD_Clear();

// Calibration

LCD_String("Calibrating...");

_delay_ms(2000);

uint32_t cal_sum = 0;

for (uint8_t i = 0; i < 32; i++) { // ????? ?? 16 ?? 32

    cal_sum += ADC_Get_Value();

    _delay_ms(50);

}

base_value = (uint16_t)(cal_sum / 32);

filtered_value = base_value;

LCD_Clear();

while (1) {

    if (!(PIND & (1 << PD2))) {

        base_value = filtered_value;

```

```

    LCD_Clear();

    LCD_Gotoxy(0, 0);

    LCD_String("Zeroing...");

    _delay_ms(500);

    LCD_Clear();
}

current_value = ADC_Get_Value();

// IIR Filter

filtered_value = (filtered_value * 5 + current_value * 5) / 10;

int16_t diff = (int16_t)filtered_value - (int16_t)base_value;

DetectionState new_pending;

// Ferrous

if (diff < -60) {

    new_pending = STATE_FERROUS;

}

// Non-Ferrous

else if (diff < -15) {

    new_pending = STATE_NON_FERROUS;

}

// Clear

else if (abs(diff) < THRESHOLD_EXIT) {

    new_pending = STATE_CLEAR;

}

// Keep Previous

else {

    new_pending = current_state;

}

if (new_pending == pending_state) {

    if (confirm_counter < CONFIRM_COUNT) confirm_counter++;

} else {

```

```

    pending_state = new_pending;

    confirm_counter = 1;
}

if (confirm_counter >= CONFIRM_COUNT) {

    current_state = pending_state;
}

if (current_state == STATE_CLEAR) {

    update_baseline(&base_value, filtered_value);
}

sprintf(buffer, "D:%-4d B:%-4u ", diff, base_value);

LCD_Gotoxy(0, 0);

LCD_String(buffer);

LCD_Gotoxy(1, 0);

switch (current_state) {

    case STATE_FERROUS:

        LCD_String("Iron/Steel Det !");

        break;

    case STATE_NON_FERROUS:

        LCD_String("Copper/Alum Det!");

        break;

    default:

        LCD_String("Clear... ");

        break;

}

    _delay_ms(50);
}

return 0;
}

```

6. Conclusion

In this project, a dual-coil Induction Balance (IB) metal discriminator centered around the ATmega32A microcontroller was successfully simulated and analyzed. The implementation of an isolated Reference Coil in conjunction with the active Sense Coil proved to be an effective engineering approach to mitigate environmental and thermal drift, significantly improving the baseline stability of the system.

While hardware constraints—primarily the non-sinusoidal nature of the PWM excitation signal and the fundamental quadrant limitations of the XOR logic gate—restricted high-resolution discrimination between similar non-ferrous metals like Copper and Aluminum, the system demonstrated high sensitivity in detecting metal presence and distinguishing broad material categories. By deploying the Amplitude-Based processing firmware, a reliable and stable operational baseline was achieved. This project lays a solid foundational framework for low-cost, digital inductive sensing and provides a clear technical roadmap for future high-precision analog and digital structural optimizations.

7. System Problems & Optimization Summary Matrix

Imprecise Discrimination: Both Iron and Aluminum return negative delta values (-80 and -40), making the system read the absolute magnitude of coupling rather than a clear directional phase vector.	• Sweep and test at multiple distinct excitation frequencies to map material-specific impedance curves. • Invert the physical winding orientation or wiring terminals of the Sense Coil.	• Replace the hardware XOR gate with an active, dedicated analog Phase Detector integrated circuit (IC) capable of preserving phase quadrant sign information.
Excitation Frequency Disalignment: The baseline 10 kHz operating frequency is not at the ideal crossover threshold where phase characteristics invert cleanly (causing Iron to lag and Copper to lead).	• Systematically sweep the frequency range by adjusting the Output Compare Register (OCR0) to fixed lower/higher values: OCR0 = 90, 80, 70, or 60.	• Isolate the exact register value where Iron consistently drives the signal in one polarity direction while Copper and Aluminum drive it in the opposite direction.
Functional Bottlenecks of the XOR Gate: The 74HC86 XOR gate operates strictly as a magnitude phase-difference indexer, losing the direction of the phase vector (whether the signal is leading or lagging).	• Calibrate tight ADC firmware thresholds around a highly precise, pre-tuned analog null center.	• Upgrade the phase-demodulation stage by integrating a Synchronous Demodulator circuit, an active analog Mixer/Multiplier topology, or an AD630 High-Precision Balanced Modulator/Demodulator chip.

